



Guía de desarrollo en arquitectura 2.1



Índice

1	Objeto	3
2	Antes de empezar	4
3	Stack tecnológico	5
3.1	Angular 13	5
3.2	TypeScript 4.5.4	5
3.3	Webpack 5	5
3.4	RxJs	5
4	Arquitectura	6
4.1	Carcasa	6
4.2	Microfront	9
4.3	DKV Core	10
4.4	DKV Styles	11
5	Estrategia de desarrollo	12
5.1	Semilla	12
5.2	Estructura de módulos	16
5.3	Comunicación	17
5.4	Nginx, environments y settings	19
5.5	Despliegues	23

1 Objeto

El propósito de este documento es explicar los distintos sistemas y librerías que componen la arquitectura de carcasas v2.1 y, en base a estos elementos, establecer una estrategia de desarrollo eficaz de nuevos microfronts que saquen partido a las funcionalidades de esta arquitectura.

2 Antes de empezar

Esta guía está planteada para que el desarrollo **parta del uso de la semilla de microfront** preparada para esta arquitectura. Esta semilla contiene todos los servicios, funcionalidades y sistemas necesarios para integrarse en esta arquitectura reduciendo en varias jornadas el tiempo que habría que dedicarle si se partiese de un proyecto angular estándar.

Por tanto, la información aquí contenida asume que es de la semilla de donde se va a partir. Si por el motivo que sea se decide no usar la semilla (lo cual desaconsejamos por completo), la información aquí contenida seguiría siendo útil, pero **será inevitable tener que dedicar tiempo adicional a adaptar el proyecto base** que se use para poder integrarlo en la carcasa.

La semilla se puede clonar aquí: https://dkvnet.visualstudio.com/STS_Architecture/git/dkv.web.seed.microfront2.1

3 Stack tecnológico

Las librerías clave que componen esta arquitectura y que habría que estudiar de cara a cualquier cambio de versión son las siguientes:

3.1 Angular 13

Esta arquitectura está diseñada y ejecutada en **Angular 13**. Si bien es cierto que la versión de angular no debería estar fuertemente acoplada a la solución adoptada por la arquitectura, hay pilares que hay que tener en cuenta de cara a posibles migraciones, puesto que nuevas versiones podrían causar incompatibilidades o problemas inesperados.

3.2 TypeScript 4.5.4

Generalmente es posible y aconsejable usar versiones más nuevas de este lenguaje, pero teniendo en cuenta que **no se trataría de un cambio trivial** – podrían surgir problemas de compilación y de tipado que habría que resolver con cuidado.

3.3 Webpack 5

La integración de carcasa y microfronts se realiza mediante la generación de módulos por Webpack. De todo el stack tecnológico, **esta es la pieza fundamental** y la que más atención requeriría de cara a posibles cambios de versión, de esta librería o de angular.

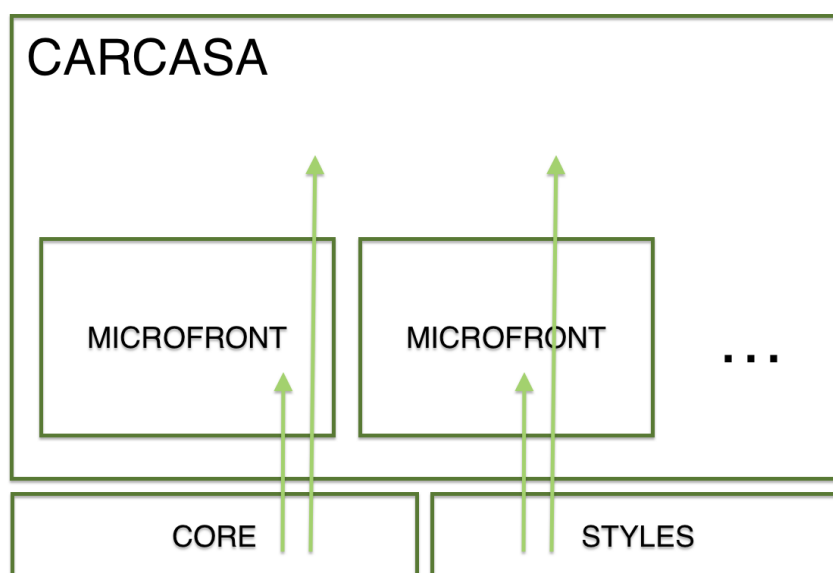
3.4 RxJs

Esta librería se utiliza para gestionar **toda clase de asincronía** – tanto a nivel de capa de APIs, como a comunicación asíncrona entre componentes – como para gestionar la comunicación entre carcasa y microfronts. Esta librería también es una pieza importante dentro de la arquitectura y es particularmente sensible a cambios de versión.

4 Arquitectura

Para poder desarrollar nuevas aplicaciones para DKV hay que entender con qué arquitectura se está trabajando, **puesto que el diseño y ejecución de nuevos desarrollos va a estar inevitablemente condicionado y favorecido por la forma de trabajar que conlleva este diseño.**

La arquitectura de las aplicaciones de negocio de DKV está basada en los **microfronts**. Esencialmente esta arquitectura deshace una aplicación monolítica tradicional en subsistemas más pequeños (microfronts) de forma **similar a una arquitectura de microservicios**, y sirve estos módulos más pequeños dentro de un paraguas común: la carcasa.



Generalmente, se tiene **una carcasa por cada área de negocio** de DKV (comercial, salud, clientes, empresas...) **que albergan los distintos procesos y servicios en aplicaciones independientes, los microfronts** (reembolsos y autorizaciones para la carcasa de salud, leads y colectivos para la carcasa comercial, etc.)

A continuación, se detalla a nivel técnico cada uno de los actores que intervienen en esta arquitectura:

4.1 Carcasa

La carcasa es esencialmente un proyecto – contenedor que apenas contiene lógica de negocio. **Es en el puerto en el que levantemos este proyecto en el que vamos a trabajar y desarrollar todas nuestras aplicaciones**, por defecto es el puerto 8081.

Esencialmente la carcasa se encarga de aspectos relacionados con la **autenticación del usuario, la gestión de esta información con los microfronts y cuestiones de navegación y enrutado, así como el estado compartido de la aplicación.**

Para poder cargar aplicaciones en local, la carcasa utiliza el archivo local-microfronts.json para poder mapear los posibles microfronts que puede cargar a su menú lateral, desde donde accederemos al resto de desarrollos. Este archivo exporta un array de objetos con esta forma:

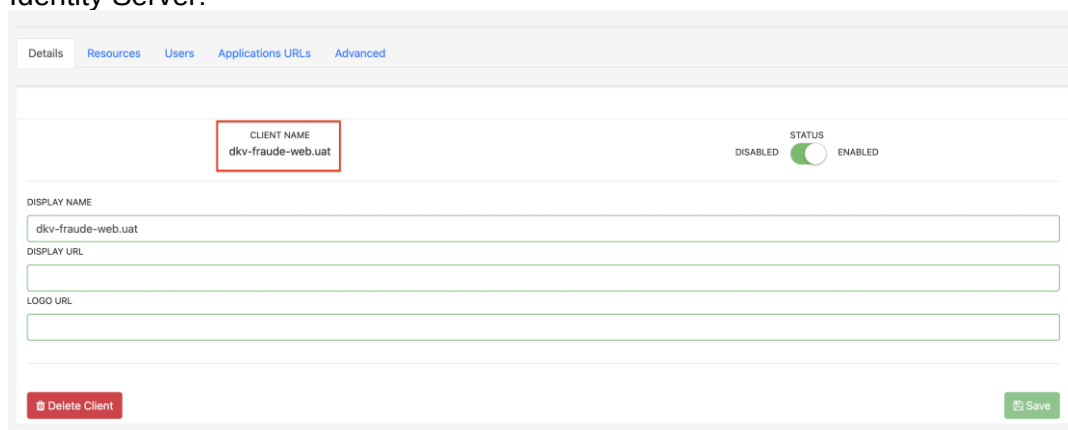
```

{.} local.microfronts.json M X
src > assets > clients > {.} local.microfronts.json > {} 8
109 {
110   "clientId": "dkv-hiperfrecuentacion-web.uat",
111   "remoteEntry": "http://localhost:9001/remoteEntry.js",
112   "remoteName": "dkv-web-hiperfrecuentacion",
113   "routerLink": "hiperfrecuentacion",
114   "label": "Hiperfrecuentación",
115   "expanded": false,
116   "styleClass": "",
117   "icon": "dkv-icon-document_1",
118   "items": []
119 },
120 {
121   "clientId": "dkv-personaldoctor-web.dev",
122   "remoteEntry": "https://personaldoctor.dev.kubedev01.dkvnet.com/remoteEntry.js",
123   "remoteName": "dkv-web-personaldoctor",
124   "routerLink": "personaldoctor",
125   "label": "Personal Doctor",
126   "expanded": false,
127   "styleClass": "",
128   "icon": "dkv-icon-builder",
129   "items": []
130 },
131 {
132   "clientId": "dkv-fraude-web.uat",
133   "remoteEntry": "http://localhost:9001/remoteEntry.js",
134   "remoteName": "dkv-web-fraude",
135   "routerLink": "fraude",
136   "label": "Fraude",
137   "expanded": false,
138   "styleClass": "",
139   "icon": "dkv-icon-document",
140   "items": []
141 }
142 ]

```

De aquí es importante entender los siguientes aspectos:

- El **client-id** se debe corresponder con el client-id que demos a nuestra aplicación en Identity Server:



- El **client-id** indica en que entorno vamos a poder ver este microfront: por ejemplo, dkv-fraude-web.uat lo podremos ver en UAT porque así lo indica el client-id. Es necesario que el entorno de la carcasa, el local.microfronts.json y el microfront estén alineados. Para cambiar los entornos de carcasa o microfront, abrimos el archivo environments.ts y elegimos el entorno al que queremos apuntar:

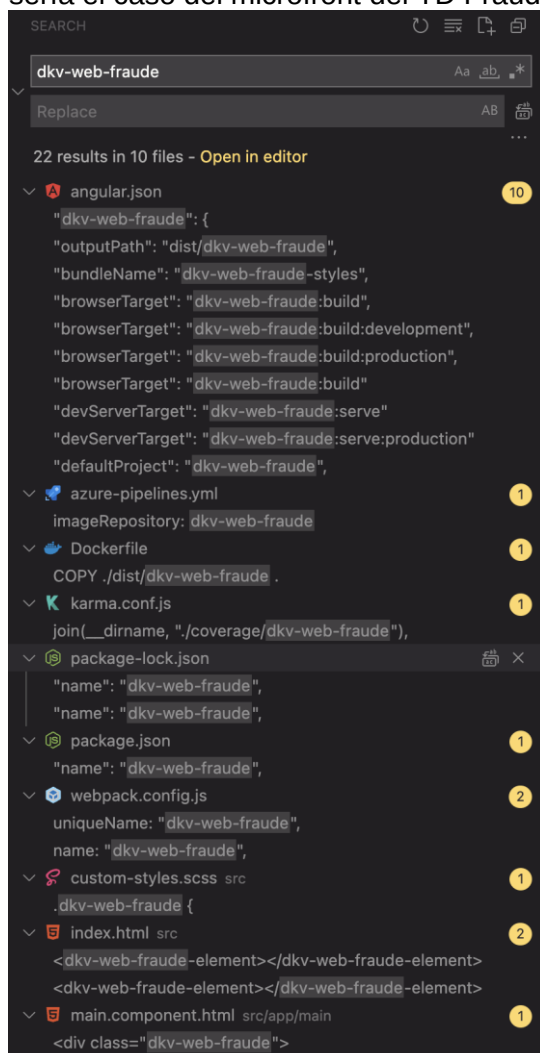
```

src > environments > environment.ts > ...
4
5  /* DEV */
6  // export const environment = {
7  //   production: false,
8  //   loadRemoteSettings: false,
9  //   enableAuth: true,
10 //   enableAcquire: true,
11
12 //   envName: 'local', // IMPORTANTE: siempre 'local'
13
14 //   contextoBaseUrl: 'https://apim-dev.dkvseguros.com/ctx',
15 //   contextoSubscriptionKey: 'e4754015c25b456699cddb037d2c436f',
16 //   contextoClientID: 'CTX_SALUD_DEV',
17 //   authSettings: {
18 //     authority: 'https://idsrv4.dev.kubedev01.dkvnet.com',
19 //     clientId: 'dkv-salud-shell.dev',
20 //     scope: 'openid profile ExtendedProfile dkv-contexto-api.dev',
21 //     id: ''
22 //   },
23 // };
24
25 /* UAT */
26 export const environment = {
27   production: false,
28   loadRemoteSettings: false,
29   enableAuth: true,
30   enableAcquire: true,
31
32   envName: 'local', // IMPORTANTE: siempre 'local'
33
34   contextoBaseUrl: 'https://apim-pre.dkvseguros.com/ctx',
35   contextoSubscriptionKey: '7b0a780019354ef2844d6b0c356f361f',
36   contextoClientID: 'CTX_SALUD_UAT',
37   authSettings: {
38     authority: 'https://idsrv4.uat.kubepro01.dkvnet.com',
39     clientId: 'dkv-salud-shell.uat',
40     scope: 'openid profile ExtendedProfile dkv-contexto-api.uat',
41     id: ''
42   },
43 };

```

- En el campo **remoteEntry** especificamos el puerto en el que vamos a levantar el microfront, que es donde se creará el archivo remoteEntry.js que nos permitirá enganchar microfront y carcasa. Por defecto este puerto será el 9001, aunque se puede cambiar y tener dos aplicaciones levantadas al mismo tiempo.

- El remoteName debe corresponderse con el nombre que se dé al microfront. Este sería el caso del microfront del TD Fraude, cuyo nombre es dkv-web-fraude:



4.2 Microfront

El microfront es un proyecto independiente que se consume desde la carcasa. Para ello se emplea la tecnología de **module federation** de Webpack, que permite compilar un proyecto como un módulo que se puede consumir desde una aplicación externa, en nuestro caso la carcasa. Este módulo se compila y se expone mediante un archivo .js en la ruta /remoteEntry.js, que es donde se enganchará la carcasa para poder mostrar nuestra aplicación.

Este archivo se genera automáticamente al ejecutar npm run start.

De cara a un nuevo desarrollo no debería modificarse el archivo webpack.config.js salvo para incluir el nombre correcto del microfront:

```

webpack.config.js ×
webpack.config.js > ...
You, 3 months ago | 1 author (You)
1 const ModuleFederationPlugin = require("webpack/lib/container/ModuleFederationPlugin");
2
3
4 module.exports = {
5   output: {
6     publicPath: "auto",
7     uniqueName: "dkv-web-fraude",
8   },
9   optimization: {
10    // Only needed to bypass a temporary bug
11    runtimeChunk: false,
12  },
13  experiments: {
14    outputModule: true,
15  },
16  plugins: [
17    new ModuleFederationPlugin({
18      library: { type: "module" },
19
20      // For remotes (please adjust)
21      name: "dkv-web-fraude",
22      filename: "remoteEntry.js",
23      exposes: {
24        "./web-components": "./src/bootstrap.ts", // bootstrap --> main --> AppModule --> WebComp
25      },
26      shared: {
27        "@angular/core": {
28          singleton: false,
29          strictVersion: true,
30          requiredVersion: "13.1.1",
31        },
32        "@angular/common": {
33          singleton: false,
34          strictVersion: true,
35          requiredVersion: "13.1.1",
36        },
37        "@angular/router": {
38          singleton: false,
39          strictVersion: true,
40          requiredVersion: "13.1.1",
41        },
42      },
43    }),
44  ],
45 };

```

Por lo demás, si se parte de la semilla, no deberían ser necesarios más cambios en el microfront para poder levantarlo desde carcasa y empezar a desarrollar, salvo lo mencionado anteriormente en la sección de la carcasa: es necesario alinear los entornos de todos los aplicativos y que los nombres del microfront y de su entrada en el local.microfronts.json se correspondan.

4.3 DKV Core

https://dkvnet.visualstudio.com/STS_Componentes_Everis/_git/dkv.core

DKV Core es una librería transversal que contiene clases y servicios usados tanto en carcasa y microfront, extraídos a una librería para no tener código duplicado.

Estos servicios gestionan principalmente la autenticación e información del usuario y se debe ser consciente que cualquier cambio a estas clases tendría un impacto en todas las carcasas y microfronts que consumiesen la librería, que será un porcentaje cercano al 100%

Más allá de esto, se debe tener en cuenta que si en un futuro se debe implementar una nueva clase o servicio que se consuma desde carcasa y microfront, o desde dos o más microfronts, **debe ser movido a esta librería para no hacer ni tener que mantener código duplicado.**

4.4 DKV Styles

https://dkvnet.visualstudio.com/STS_Componentes_Everis/_git/dkv.ui.componentes.3.0

Al igual que DKV Core, DKV Styles es una librería transversal que se consume tanto desde los microfronts como desde la carcasa.

Esta librería **solo exporta estilos CSS**, pero si se estudia el repositorio referenciado, se comprobará que es un proyecto angular completo. Se trata del antiguo Showcase de la arquitectura 1.0 migrado a la arquitectura 2.1.

De esta librería hay que tener claro lo siguiente:

- Solo se debe consumir y exportar estilos CSS
- Se puede levantar en local para probar los estilos
- Ningún otro componente, servicio o sistema en este repositorio debe ser exportado o utilizado, solo los estilos CSS

5 Estrategia de desarrollo

5.1 Semilla

Como se establece al inicio del documento, esta guía de desarrollo de microfronts está pensada para que se parta de la semilla que se puede clonar aquí: https://dkvnet.visualstudio.com/STS_Architecture/_git/dkv.web.seed.microfront2.1

Esta semilla está equipada con todos los servicios, funcionalidades y sistemas necesarios **para poder integrarse con mínimo esfuerzo en la carcasa.**

Si se necesitase usar una versión superior de angular, esta semilla debería ser relativamente fácil de migrar, siendo conscientes que **también sería necesario tener una versión del DKV Core y probablemente DKV Styles en la misma versión.**

Para empezar el desarrollo, hay que hacer una serie de tareas. Estas tareas están enumeradas y descritas en el README.md:

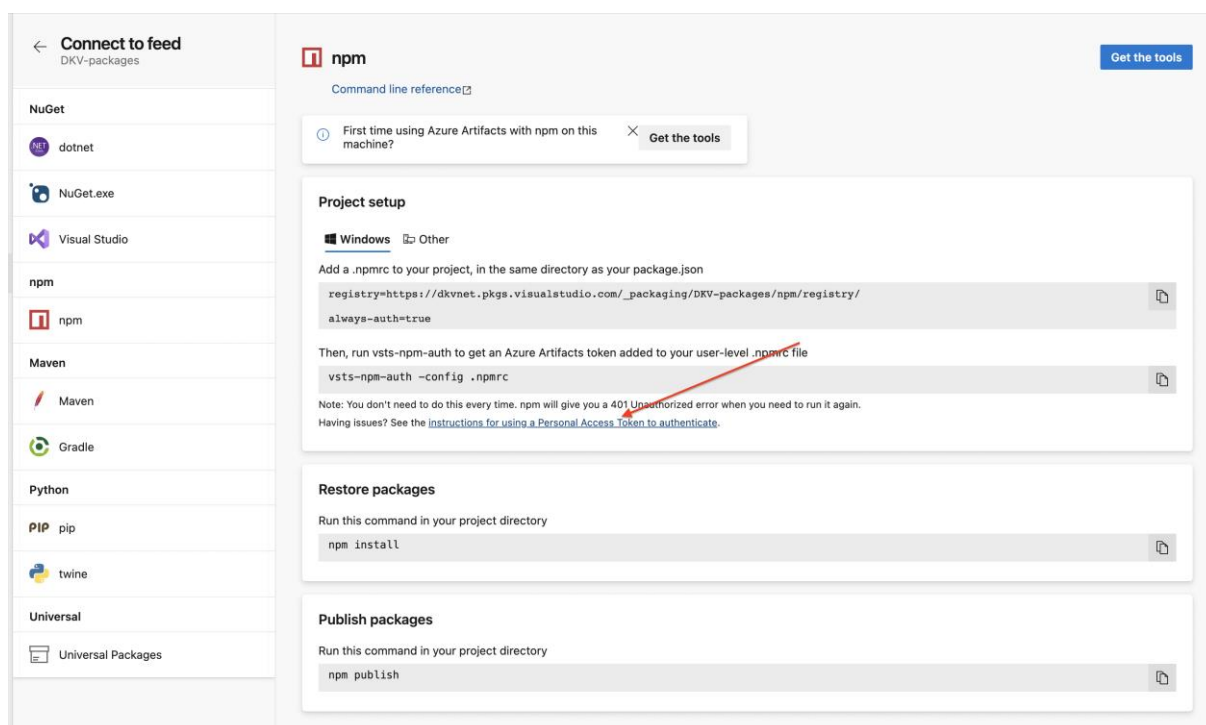
First steps:

1. After clone this repo, remove the .git folder, create a new one with ``git init`` and add your remote repository ``git add remote origin <url_of_your_repo>``
2. Generate [Azure Artifacts token](https://dkvnet.visualstudio.com/STS_Componentes_DKV/_artifacts/feed/DKV-packages/connect/npm) and replace .npmrc file to access DKV packages.
3. Download package dependencies "with npm install".
4. Replace all the occurrences of "dkv-web-microfront" with "dkv-web-<your_project_name>".
5. Open app.module change the value of the constants "APP_TITLE" (with the title of your microfront) and "MAIN_ROUTE_MFE" (with the root route name of your microfront). But don't add the routes of your microfront there, they must be in "main-routing.module.ts".
6. Remove "Example" module and component.
7. Search all the occurrences of "apiUrlExample" and replace/add/delete them depending of how many api's are you going to use.
8. Add Identity Server "authority, clientId and scopes" in the "authSettings" section of the settings and environments.
9. If microfront runs in standalone mode add "@dkv/styles" dependence if necessary.
10. Adapt Udate README.md and CHANGELOG.md to your project.
11. Global search of "TODO" to check features.

Los primeros tres pasos son básicos para cualquier otro proyecto; configurar correctamente el upstream de Git, generar un token .npmrc nuevo para poder descargar todas las dependencias, y descargar dichas dependencias.

El token de .nprmc es necesario porque las librerías de DKV (Styles, Core y cualquier otra) son librerías privadas a las que hay que estar suscritos mediante el Azure Devops para

poder descargarlas y hacer uso de ellas, de lo contrario recibiremos un error 401 al hacer npm install. En el link provisto en el README se indica como generar este token:



Esta es la apariencia que deberá tener el .npmrc:

```
1 registry=https://dkvnet.pkgs.visualstudio.com/_packaging/DKV-packages/npm/registry/
2 always-auth=true
3
4 ; begin auth token
5 //dkvnet.pkgs.visualstudio.com/_packaging/DKV-packages/npm/registry/:username=dkvnet
6 //dkvnet.pkgs.visualstudio.com/_packaging/DKV-packages/npm/registry/:_password=b3ZuaDVnamdqaXlibHBuNGJ0eHlxcmczcjr1bmludndlaDZzdXpsZTVtdjN4dHZwemF6cQ==
7 //dkvnet.pkgs.visualstudio.com/_packaging/DKV-packages/npm/registry/:email=npm requires email to be set but doesn't use the value
8 //dkvnet.pkgs.visualstudio.com/_packaging/DKV-packages/npm/:username=dkvnet
9 //dkvnet.pkgs.visualstudio.com/_packaging/DKV-packages/npm/:_password=b3ZuaDVnamdqaXlibHBuNGJ0eHlxcmczcjr1bmludndlaDZzdXpsZTVtdjN4dHZwemF6cQ==
10 //dkvnet.pkgs.visualstudio.com/_packaging/DKV-packages/npm/:email=npm requires email to be set but doesn't use the value
11 ; end auth token
```

Los siguientes pasos que se describen son mecánicos y consisten en configurar aspectos básicos del proyecto, como dotarlo de un nombre propio, establecer la ruta base del microfront por el cual se van a acceder al resto de rutas y quitar módulos y código boilerplate o de ejemplo.

Los pasos 8, 9 y 10 no son obligatorios o se pueden hacer más tarde.

Una vez tengamos los pasos anteriores hechos, tenemos que hacer dos cosas más antes de poder ejecutar y ver el microfront en local dentro de una carcasa:

- 1- Crear claim en Identity Server Admin – basta con darle el mismo nombre que tiene el mismo nombre que tiene el microfront en nuestro proyecto añadiendo {nombre_entorno}, y añadiendo los recursos básicos que consideremos. En caso de duda, copiad este ejemplo:

SEARCH

dkv-web-fraude

Replace

22 results in 10 files - Open in editor

angular.json

dkv-web-fraude": {

outputPath: dist/dkv-web-fraude",

bundleName: "dkv-web-fraude-styles",

browserTarget: "dkv-web-fraude:build",

browserTarget: "dkv-web-fraude:build:development",

browserTarget: "dkv-web-fraude:build:production",

browserTarget: "dkv-web-fraude:build"

devServerTarget: "dkv-web-fraude:serve"

devServerTarget: "dkv-web-fraude:serve:production"

defaultProject: "dkv-web-fraude",

webpack.config.js X

webpack.config.js > ...

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

const ModuleFederationPlugin = require("webpack/lib/container/ModuleFederationPlugin");

module.exports = {

output: {

publicPath: "auto",

uniqueName: "dkv-web-fraude",

},

optimization: {

// Only needed to bypass a temporary bug

runtimeChunk: false,

},

experiments: {

outputModule: true,

},

plugins: [

new ModuleFederationPlugin({

library: { type: "module" },

},

IdentityServer Admin

Language soporte.dkv@babel.es, Soporte

Users

Groups

Roles

Clients

Companies

Resources

Configuration

Home / Clients

5 per page

+

Add Client

Name	
identity-admin-ui.api.dev.web	
dkv-reembolsos-web.dev	
plainconcepts.pre.portal	
pc.pre.portal	
pc.dev.portal	

<

1

2

3

4

>

IdentityServer Admin

Language soporte.dkv@babel.es, Soporte

Users

Groups

Roles

Clients

Companies

Resources

Configuration

Home / Clients

Create a new client

CLIENT ID

dkv-fraude-web.dev

CLIENT NAME

dkv-fraude-web.dev

Cancel

Save & Configure

IdentityServer Admin

Language soporte.dkv@babel.es, Soporte

Users

Groups

Roles

Clients

Companies

Resources

Configuration

Home / Clients

Details Resources Users Applications URLs Advanced

Identity Resources Api Resources

Available

Assigned

→

←

☐ Email
 ☐ Extended profile
 ☐ Profile
 ☐ OpenId

IdentityServer Admin

Language ▼ soporte.dkv@babel.es, Soporte ▼

Users

Groups

Roles

Clients

Companies

Resources ▼

Configuration ▼

Home / Clients

Details

Resources

Users

Applications URLs

Advanced

Identity Resources

Api Resources

Available

Assigned

IdentityServer Admin

Language ▼ soporte.dkv@babel.es, Soporte ▼

Users

Groups

Roles

Clients

Companies

Resources ▼

Configuration ▼

Home / Clients

Details

Resources

Users

Applications URLs

Advanced

Assign Clients

Available

Assigned

IdentityServer Admin

Language ▼ soporte.dkv@babel.es, Soporte ▼

Users

Groups

Roles

Clients

Companies

Resources ▼

Configuration ▼

Home / Clients

Details

Resources

Users

Applications URLs

Advanced

Callbac URLs

Signout URL

CORS

- 2- El siguiente paso será configurar el claim en el archivo `local.microfronts.json` de la carcasa para poder ver el proyecto en local a través de la misma. Siguiendo el ejemplo de fraude, tendría que ser algo así, sustituyendo el nombre y las rutas de fraude por los que correspondan para el nuevo proyecto:

```

{..} local.microfronts.json M X
src > assets > clients > {..} local.microfronts.json > {} 8
131 {
132   "clientId": "dkv-fraude-web.uat",
133   "remoteEntry": "http://localhost:9001/remoteEntry.js",
134   "remoteName": "dkv-web-fraude",
135   "routerLink": "fraude",
136   "label": "Fraude",
137   "expanded": false,
138   "styleClass": "",
139   "icon": "dkv-icon-document",
140   "items": []
141 }
142 ]

```

Una vez hecho todos estos pasos, ejecutando `npm run start` en el microfront y la carcasa, deberíamos poder acceder a nuestro proyecto a través de la ruta `localhost:8081`.

5.2 Estructura de módulos

Solamente hay un par de puntos a tener claros en cuanto a los módulos:

El **app.module** no se debe usar como módulo principal, y no debería ser necesario modificar nada en este módulo, salvo la ruta base (`MAIN_ROUTE_MFE`) que ya se debería haber hecho en el paso anterior:


```

app.module.ts x main.module.ts main-routing.module.ts
src > app > app.module.ts > AppModule
14
    You, 3 months ago | 1 author (You)
15 @NgModule({
16   imports: [
17     BrowserModule,
18     BrowserModuleAnimationsModule,
19     CoreModule.forRoot(),
20     RouterModule.forRoot([
21       { path: '', redirectTo: MAIN_ROUTE_MFE, pathMatch: 'full' },
22       {
23         path: MAIN_ROUTE_MFE,
24         canActivate: [AuthGuard],
25         loadChildren: () => import('./main/main.module').then(m => m.MainModule)
26       },
27       {
28         path: '**',
29         redirectTo: MAIN_ROUTE_MFE
30       },
31       {
32         path: 'error',
33         loadChildren: () => import('./modules/error/error.module').then(m => m.ErrorModule)
34       },
35       {
36         path: 'unauthorized',
37         redirectTo: '/error/no-autorizado',
38         pathMatch: 'full'
39       }
40     ])
41   ],
42   declarations: [
43     AppComponent,
44   ],

```

El verdadero módulo principal es el **main.module**, el cual va a tener las rutas raíz del proyecto. En este módulo se deben importar el shared module, el cual tendrá todas los servicios o dependencias claves del proyecto para evitar importarlo múltiples veces. Si es necesario algún módulo de terceros común para todo el proyecto, como una librería de componentes, se debería importar en el shared module.

Por lo demás el main module no debería modificarse mucho más allá de las rutas que queramos definir en nuestra aplicación.

5.3 Comunicación

Microfront y carcasa cuentan con servicios de comunicación que les permiten compartir información importante entre ellos. Esencialmente, como funciona es que se crean canales a nivel de navegador con nombre (topics), y cualquier microfront o carcasa se puede suscribir a estos canales. El protocolo de comunicación se llama **PubSub** y solo contempla dos acciones: publicar información a través de un canal, y suscribirse a ese canal.

El microfront comparte con la carcasa información relativa a sus elementos de navegación (las tabs, si las hubiese) y su nombre. La carcasa comparte con el microfront información del usuario (códigos, contexto, perfil).

No debería ser necesario modificar nada de lo ya existente en los servicios de comunicación ni de microfront ni de carcasa, pues son sistemas estables y maduros y su modificación podría tener impactos no deseados en el sistema de autenticación del usuario y de la consumición de esa información.

Si fuese necesario compartir alguna información más desde el microfront a la carcasa, se haría desde este servicio, de manera análoga a como se está haciendo ya.

Simplemente habría que crear un nuevo canal y que carcasa y microfront compartiesen o gestionasen la información a través de él, respetando el orden de creación de topics que ya existe.

El servicio de comunicación del microfront:

```

A communication.service.ts ×
src > app > core > communication > A communication.service.ts > CommunicationService > initCommunication
18 constructor(
19   private userService: UserService,
20   private router: Router
21 ) { }
22
23 initCommunication() {
24   this.setShellHandshakeTopic();
25   this.setUserTopic();
26   this.setContextTopic();
27   this.setCodesTopic();
28   this.setNavigationTopic();
29   this.publishHandshake();
30 }
31
32
33 private setShellHandshakeTopic() {
34   this.handshakeSubscription = PubSub.subscribe(ShellEvents.handshake, this.handleShellHandshake.bind(this));
35 }
36
37 private setUserTopic() {
38   this.userSubscription = PubSub.subscribe(ShellEvents.user, (msg, data) => {
39     if (data !== null) this.userService.setUser(data)
40   });
41 }
42
43 private setContextTopic() {
44   this.contextSubscription = PubSub.subscribe(ShellEvents.context, (msg, data) => {
45     if (data !== null) this.userService.setContext(data)
46   });
47 }
48
49 private setCodesTopic() {
50   this.codesSubscription ? null : this.codesSubscription = PubSub.subscribe(ShellEvents.codes, this.setCodesCallback);
51 }
52
53 private setNavigationTopic() {
54   window.addEventListener(ShellEvents.navigationItems, this.setNavigationCallback)
55 }

```

El servicio de comunicación de la carcasa:

```

communication.service.ts X
src > app > core > communication > communication.service.ts > CommunicationService > setBreadcrumbTopic > PubSub.subscribe() callback
13
14 constructor(
15     private navigationService: NavigationService,
16     private userService: UserService,
17     private headerService: HeaderService,
18     private breadcrumbsService: BreadcrumbsService,
19     private alertService: AlertService
20 ) {
21     this.setMFEHandshakeTopic();
22     this.setUserNavigationItemsTopic();
23     this.setUserShellTitleTopic();
24     this.setMFEAlertTopic();
25     this.setBreadcrumbTopic()
26 }
27
28 private setMFEHandshakeTopic() {
29     PubSub.subscribe(MFEEEvents.handshake, (msg, data: MicrofrontHandshake) => {
30         if (this.currentMicroFront !== data.microfrontName) {
31             this.publishShellHandshake();
32         }
33         this.currentMicroFront = data.microfrontName;
34     });
35 }
36
37 private setUserNavigationItemsTopic() {
38     PubSub.subscribe(MFEEEvents.navigationItems, (msg, data) => {
39         this.navigationService.emitItems(data.navigationItems);
40     });
41 }
42
43 private setUserShellTitleTopic() {
44     PubSub.subscribe(MFEEEvents.shellTitle, (msg, data) => {
45         console.log('recieved change', data.shellTitle);
46         this.headerService.emitShellTitle(data.shellTitle)
47     });
48 }
49
50 private setMFEAlertTopic() {
51     PubSub.subscribe(MFEEEvents.alert, (msg, data) => {
52         this.alertService.showAlert(data.type, data.title, data.description)

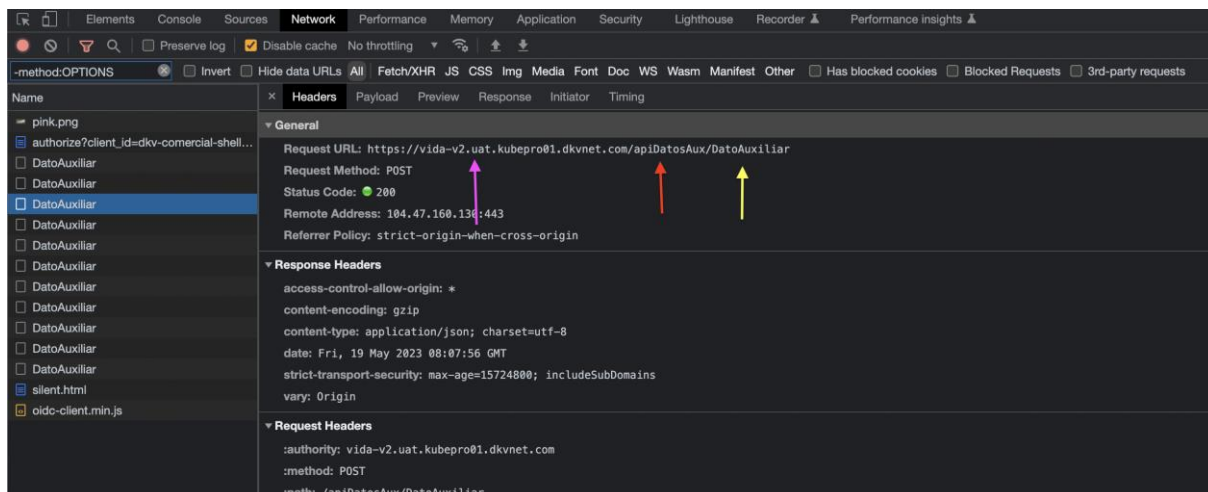
```

5.4 Nginx, environments y settings

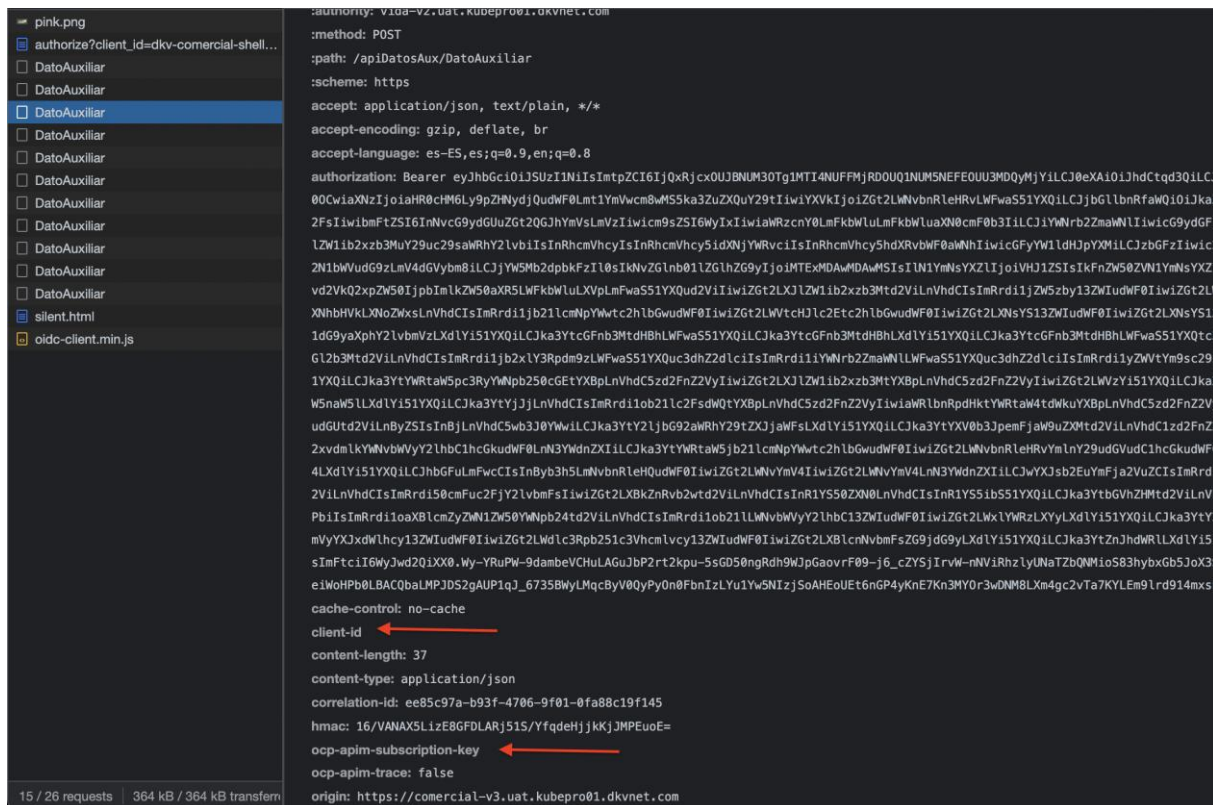
En la arquitectura 2.1, tanto la carcasa como los microfronts usan una arquitectura de llamadas a API basada en un proxy de Nginx. Lo que se logra con esta arquitectura es poder llamar directamente a los servicios de DKV del API Manager, sin comprometer la seguridad en estas llamadas al ofuscar todas las claves sensibles al navegador. De esta manera, se logra la seguridad que tendríamos usando un middleware que se encargase de la securización de las llamadas, pero sin tener que contar con esta capa adicional, con la mejora de rendimiento y menor necesidad de mantenimiento y desarrollo que esto conlleva.

Las claves que son necesarias ofuscar en todo caso son la SUBSCRIPTION KEY, CLIENT ID y la BASE URL de la llamada.

Tomemos esta llamada como ejemplo, lo primero que vemos es que desde el navegador no estamos llamando directamente al APIM, si no que llamamos a una url propia (flecha rosa), unida con un namespace (rojo) que juntos forman la ruta de Nginx donde se sustituirán las claves vacías por las claves necesarias para poder llamar al servicio. En amarillo está la parte final de la llamada a la API que se concatenará a la URL cuando las secciones anteriores se sustituyan por la URL del APIM.



También vemos que ni el navegador ni por tanto el usuario pueden ver las claves sensibles como la subscription key o el client id.



Este proxy Nginx funciona de la siguiente manera. Se configura en el archivo nginx.conf, en la carpeta /k8s/config_secrets, tiene este aspecto (nginx en la izquierda, environments.ts en la derecha):

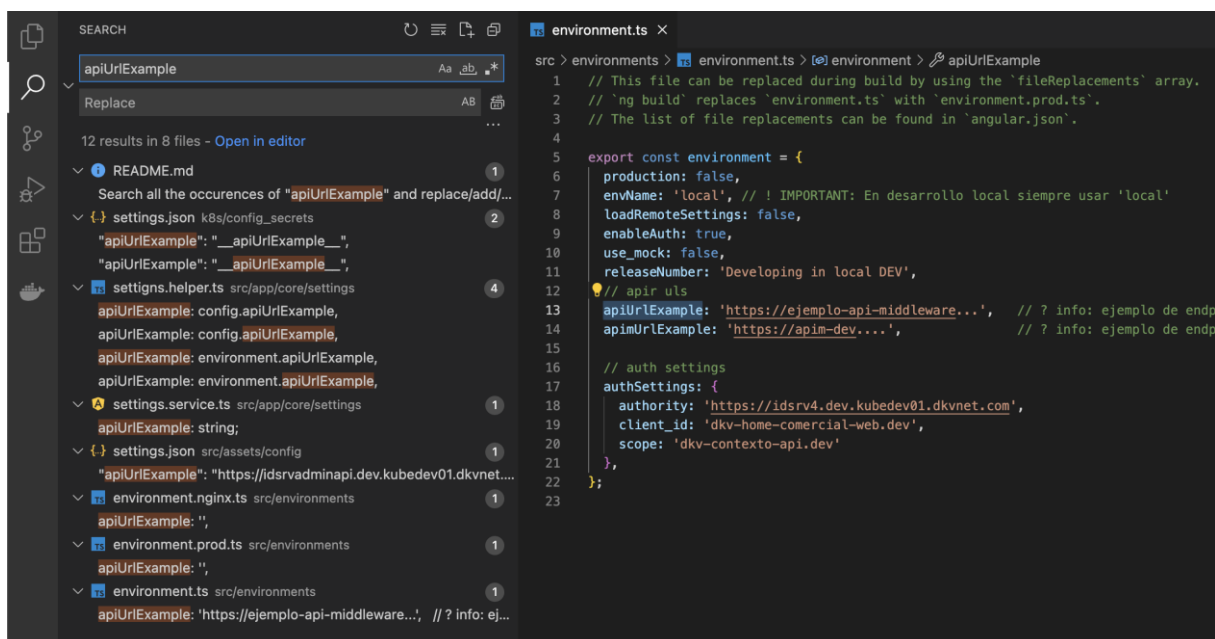
The screenshot shows the Azure DevOps web interface for a project named 'CPC_VIDA'. The 'Variables' tab is selected, showing a list of predefined variables for the pipeline 'dkv.web.vida[2.1]-CD'. The 'Pipeline variables' section is active, displaying a list of predefined variables. A red box highlights the 'API_TARIFICAR' variable, and another red box highlights the 'CLIENT_ID' variable. A red arrow points from the 'Filter by keywords' search bar to the 'UAT' dropdown menu.

Name	Value	Scope
API_AGENTE_ACTIVIO	https://apim-pre.dkvseguros.com/cpc/authmediadores/apiv1/Agente	UAT
API_CONTRATACION	https://apim-pre.dkvseguros.com/cpc/contratacion/genericv1	UAT
API_DATOS_AUX	https://apim-pre.dkvseguros.com/cpc/datosauxiliares/v1	UAT
API_DOCUMENTACION	https://apim-pre.dkvseguros.com/cpc/documentacion/v3	UAT
API_EMAIL	https://apim-pre.dkvseguros.com/cpc/emailsimulacion/v1	UAT
API_TARIFICAR	https://apim-pre.dkvseguros.com/cpc/tarifacionv2/apiv1	UAT
API_TARIFICAR_070	https://apim-pre.dkvseguros.com/cpc/tarifacionv2/apiv1	UAT
AUTH_TALENTO	63e19fd0-526e-4094-9ffe-73a8bbbdbf84	UAT
authSettings.authority	https://idsrv4.uat.kubepro01.dkvnet.com	UAT
authSettings.clientId	dkv-contratacion-web.uat	UAT
authSettings.scope	dkv-contexto-api.uat	UAT
CLIENT_ID	CONTRATACION_PRE	UAT
cluster	kubepro01.dkvnet.com	UAT
HOST	apim-pre.dkvseguros.com	UAT
k8sCorsAllowDomain	https://comercial-v3.uat.kubepro01.dkvnet.com/	UAT
k8sNamespace	uat	UAT
SHELL_DOMAIN	https://comercial-v3.uat.kubepro01.dkvnet.com	UAT
SUBSCRIPTION_KEY	9294e9df256a418888394510d06f5974;product=contratacion	UAT
SUBSCRIPTION_KEY_TARIFICAR	7a6b3f38c2b4985b0fc4fd34408929e;product=tarificador070	UAT
TUA_HMAC	1234qwer	UAT

Por lo tanto, hemos establecido que las claves deben existir, estar actualizadas y sincronizadas tanto en Azure para los entornos de despliegue, como en `environments.ts` para local, para que la aplicación pueda llamar correctamente a los servicios de DKV.

De cara a **configurar las claves de una nueva aplicación, o añadir nuevas claves**, el proceso siempre es el mismo, tanto si se parte de la semilla como si se parte de un proyecto ya en vuelo: replicar la configuración de una clave ya existente.

En el caso de la semilla, **buscaríamos las claves** de ejemplo y las sustituiríamos por claves reales, de esta manera no se nos escapa ningún archivo:



De la misma manera en un proyecto ya existente:

```

src > environments > environment.ts > API_TARIFICAR
36 // scope: 'dkv-contexto-api.dev'
37 // }
38 // };
39
40 /* UAT */
41 export const environment = {
42   production: false,
43   envName: 'local',
44   releaseNumber: '2.0.0',
45   appName: 'dkv-contratacion-web',
46   host: 'apim-pre.dkvseguros.com',
47   // apis
48   API_TARIFICAR: 'https://apim-pre.dkvseguros.com/cpc/tarificacionv2/api/v1',
49   API_TARIFICAR_070: 'https://apim-pre.dkvseguros.com/cpc/tarificacionv2/api/v1',
50   API_DATOS_AUX: 'https://apim-pre.dkvseguros.com/cpc/datosauxiliares/v1',
51   API_EMAIL: 'https://apim-pre.dkvseguros.com/cpc/emailsimulacion/v1',
52   API_STS: 'https://apim-pre.dkvseguros.com/sts/validaciones/validacion/v1',
53   API_PROYECTOS: 'https://apim-pre.dkvseguros.com/cpc/proyectosv3',
54   API_DOCUMENTACION: 'https://apim-pre.dkvseguros.com/cpc/documentacionv3',
55   API_CONTRATACION: 'https://apim-pre.dkvseguros.com/cpc/contratacion/generic/v1',
56   API_LEADS: 'https://apim-pre.dkvseguros.com/cpc/ciclovida/comercial/api/leads',
57   API_NORMALIZACION: 'https://apim-pre.dkvseguros.com/sts/normalizacion/api/v2',
58   API_SOLICITUD: 'https://apim-pre.dkvseguros.com/ccm/solicitudv2/Solicitud/v2',
59   API_TALENTO: 'https://pre-ocr.talentomobile.com/id/v5',
60   API_AGENTE_ACTIVADO: 'https://apim-pre.dkvseguros.com/cpc/authmediadores/api/v1/Agente',
61   // client id
62   CLIENT_ID: 'CONTRATACION_PRE',
63   // subscription
64   SUBSCRIPTION_KEY: '9294e9df256a418888394510d06f5974;product=contratacion',
65   SUBSCRIPTION_KEY_LEAD: '30d78578466041bdaba30863f1c16410',
66   SUBSCRIPTION_KEY_TARIFICAR: '7a6b31f38c2b4985b0fcafd34408929e;product=tarificador070',
67   AUTH_TALENTO: '63e19fd0-526e-4094-9f1e-73a8bbdbf84',

```

En el caso de añadir una nueva suscription key o nuevo client id, esto sería suficiente. Pero si queremos añadir una **nueva API**, tendremos que crear un nuevo namespace en Nginx, que tendrá que corresponderse con el nombre de la api en el environments.prod.ts:

```

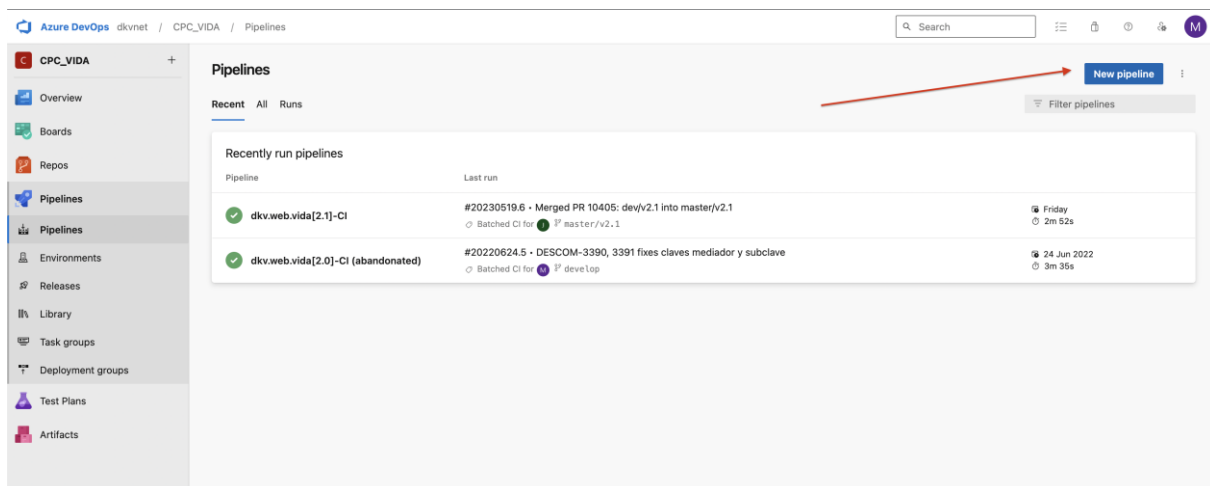
src > environments > environment.prod.ts > ...
3 // The list of file replacements can be found in 'angular.json'.
4
5 export const environment = {
6   production: true,
7   envName: 'nginix',
8   releaseNumber: '2.1.0',
9   appName: 'dkv-contratacion-web',
10  host: 'apim-dev.dkvseguros.com',
11  loadRemoteSettings: true,
12  enableAuth: true,
13  useMock: false,
14  // apis
15  API_TARIFICAR: 'apiTarificar',
16  API_TARIFICAR_070: 'apiTarificar070',
17  API_DATOS_AUX: 'apiDatosAux',
18  API_EMAIL: 'apiEmail',
19  API_STS: 'apiSts',
20  API_PROYECTOS: 'apiProyectos',

```

5.5 Despliegues

De cara a desplegar una nueva aplicación por primera vez hay **tres aspectos** que tenemos que tener preparados:

1. **Tener el client id creado y configurado en Identity Server.** Si hemos seguido la guía, esto ya estaría hecho. Si no, se describen los pasos en el punto 5.1.
2. **Configurar el azure-pipelines.yml.** El de la semilla está ya preparado, solo tenemos que asegurarnos que hemos configurado correctamente el nombre de la aplicación en la variable del **imageRepository**, que también debería estar hecho si hemos seguido los pasos del punto 5.1. En azure seleccionamos la opción de nueva pipeline y seguimos los pasos para seleccionar como plantilla el archivo azure-pipelines de nuestro repositorio:



Azure DevOps dkvnet / CPC_VIDA / Pipelines

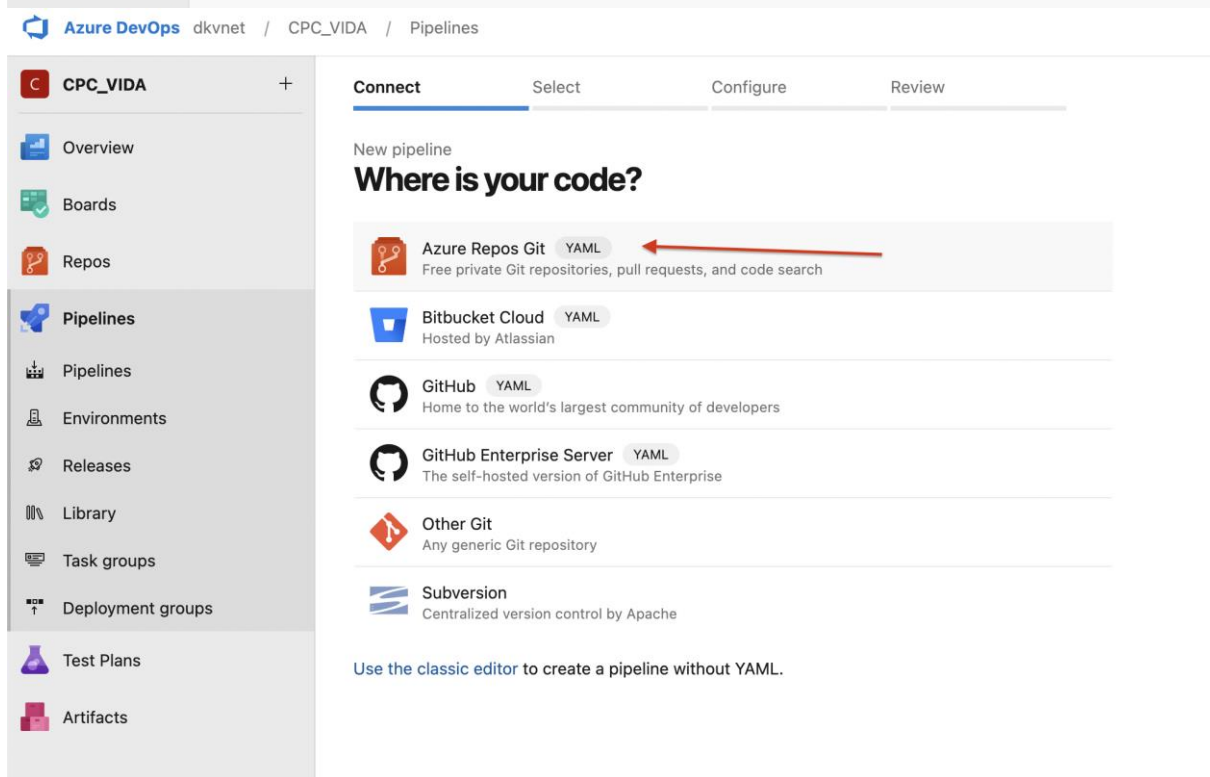
Pipelines

Recent All Runs

Filter pipelines

Recently run pipelines

Pipeline	Last run
dkv.web.vida[2.1]-CI	#20230519.6 • Merged PR 10405: dev/v2.1 into master/v2.1 Batched CI for master/v2.1 Friday 2m 52s
dkv.web.vida[2.0]-CI (abandoned)	#20220624.5 • DESCOM-3390, 3391 fixes claves mediador y subclave Batched CI for dev/Log 24 Jun 2022 3m 35s



Azure DevOps dkvnet / CPC_VIDA / Pipelines

CPC_VIDA

Overview Boards Repos Pipelines Environments Releases Library Task groups Deployment groups Test Plans Artifacts

Connect Select Configure Review

New pipeline

Where is your code?

- Azure Repos Git** **YAML**
Free private Git repositories, pull requests, and code search
- Bitbucket Cloud** **YAML**
Hosted by Atlassian
- GitHub** **YAML**
Home to the world's largest community of developers
- GitHub Enterprise Server** **YAML**
The self-hosted version of GitHub Enterprise
- Other Git**
Any generic Git repository
- Subversion**
Centralized version control by Apache

Use the classic editor to create a pipeline without YAML.

The screenshot shows the Azure DevOps 'Configure your pipeline' page. The left sidebar contains navigation links for Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, and Artifacts. The main area displays a list of pipeline templates. A red arrow points to the 'Existing Azure Pipelines YAML file' option. A modal dialog is open, titled 'Select an existing YAML file', showing a dropdown for 'Branch' (main) and a text input for 'Path' (azure-pipelines.yml).

- Una vez la pipeline de integración se haya ejecutado correctamente, tendremos que **configurar la pipeline de despliegue**. Tendremos que seleccionar el artefacto creado por la primera pipeline y configurar todas las claves de las que hablamos en el punto 5.4, así como variables de Release y de DEV para un primer despliegue. En caso de dudas respecto a variables de Release, la mejor idea es replicar la

configuración que se ha ejecutado en casi cualquier otra aplicación en lo que a variables de Release respecta:

Azure DevOps dkvnet / CPC_VIDA / Pipelines / Releases

Live updates are not supported in this view for old domain URLs. Move to new Azure DevOps URLs

Search all pipelines

+ New

+ New release pipeline

Import release pipeline

dkv.web.vida[2.0]-CD-deprecat...

UAT

dkv.web.vida[2.1]-CD

Releases Deployments Analytics

Pending approval on PRO stage.

Releases

Release-298

20230519... master/v2.1

Release-297

20230519... dev/v2.1

Release-296

20230519... master/v2.1

Release-295

20230519... dev/v2.1

Release-294

20230519... master/v2.1

Release-293

20230519... dev/v2.1

Release-292

Azure DevOps dkvnet / CPC_VIDA / Pipelines / Releases

Live updates are not supported in this view for old domain URLs. Move to new Azure DevOps URLs

Search all pipelines

+ New

dkv.we + New release pipeline

PRO Import release pipeline

dkv.web.vida[2.0]-CD-deprecat... UAT

dkv.web.vida[2.1]-CD

Releases Deployments Analytics

Pending approval on PRO stage.

Releases

- Release-298 20230519... master/v2.1
- Release-297 20230519... dev/v2.1
- Release-296 20230519... master/v2.1
- Release-295 20230519... dev/v2.1
- Release-294 20230519... master/v2.1
- Release-293 20230519... dev/v2.1
- Release-292

Azure DevOps dkvnet / CPC_VIDA / Pipelines / Releases

All pipelines > New release pipeline

Pipeline Tasks Variables Retention Options History

Artifacts | + Add

Stages | + Add

+ Add an artifact

Schedule not set

Stage 1 Select a template

Select a template

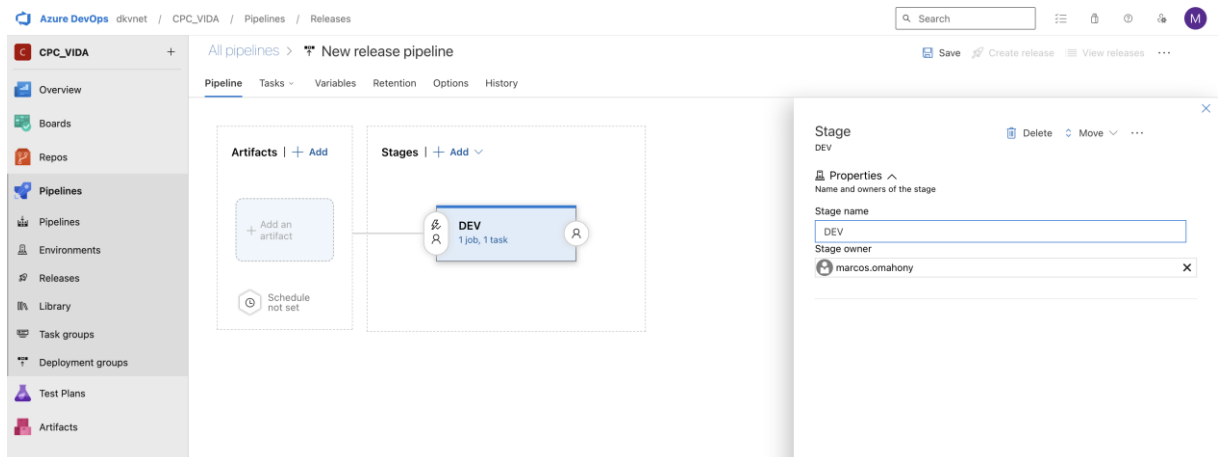
Or start with an Empty job

Featured

- Azure App Service deployment
- Deploy a Java app to Azure App Service
- Deploy a Node.js app to Azure App Service
- Deploy a PHP app to Azure App Service and Azure Database for MySQL
- Deploy a Python app to Azure App Service and Azure database for MySQL
- Deploy to a Kubernetes cluster
- IIS website and SQL database deployment

Others

- Azure App Service deployment with continuous monitoring
- Azure App Service deployment with slot
- Azure App Service deployment with tests and performance tests



Azure DevOps dkvnet / CPC_VIDA / Pipelines / Releases

All pipelines > New release pipeline

Pipeline Tasks Variables Retention Options History

Artifacts | + Add

Stages | + Add

DEV 1 job, 1 task

Stage

DEV

Properties

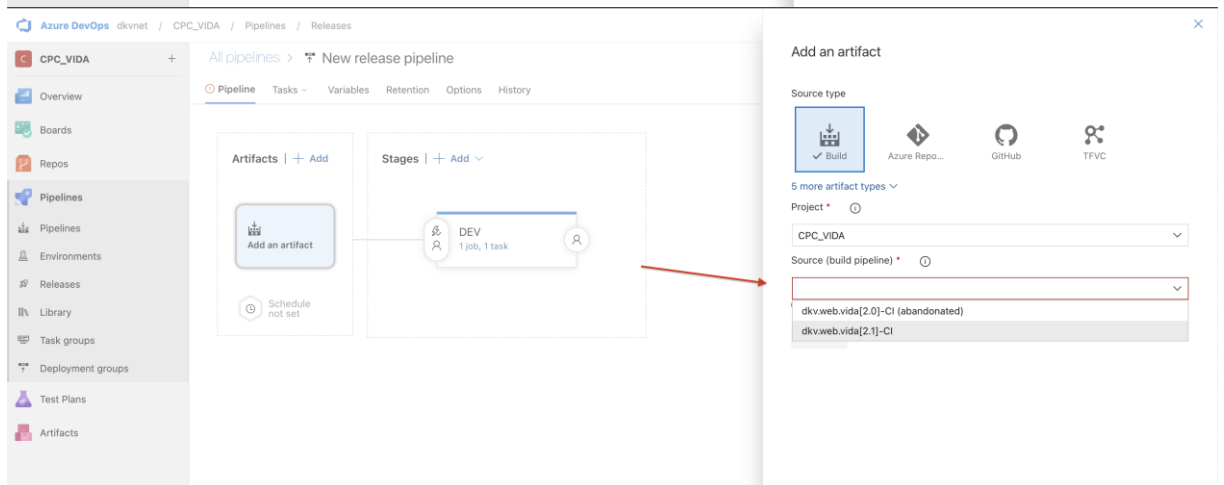
Name and owners of the stage

Stage name

DEV

Stage owner

marcos.omahony



Azure DevOps dkvnet / CPC_VIDA / Pipelines / Releases

All pipelines > New release pipeline

Pipeline Tasks Variables Retention Options History

Artifacts | + Add

Stages | + Add

DEV 1 job, 1 task

Add an artifact

Source type

Build

Azure Repo...

GitHub

TFVC

5 more artifact types

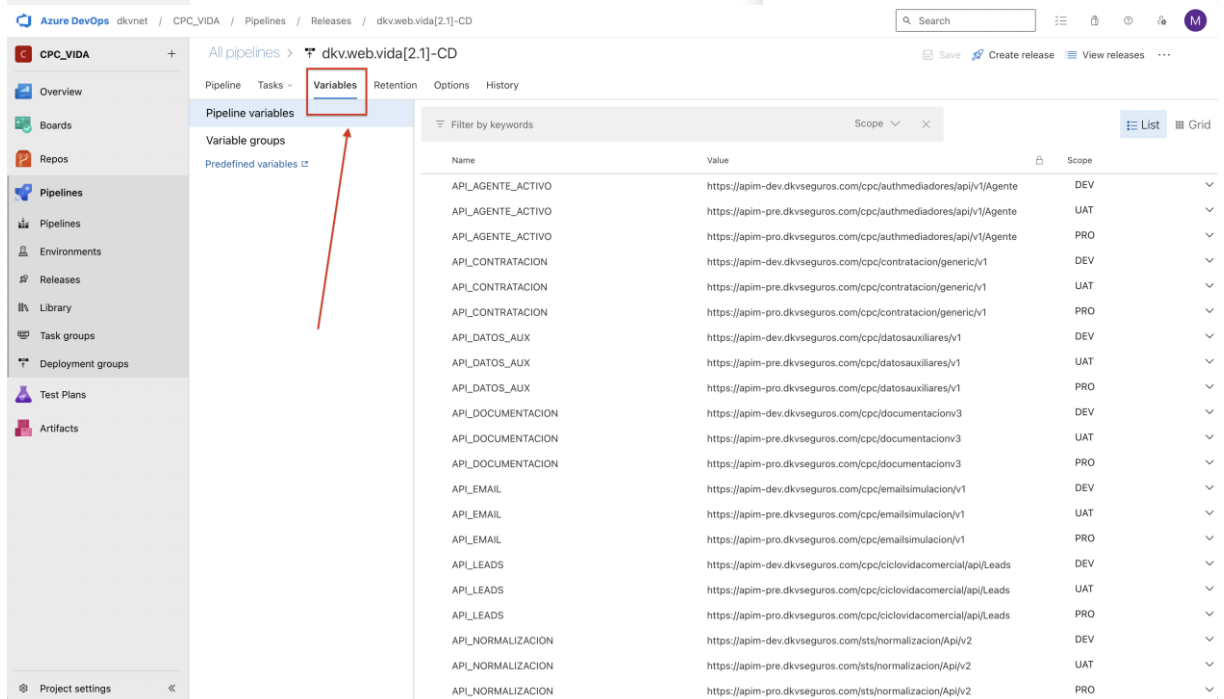
Project

CPC_VIDA

Source (build pipeline)

dkv.web.vida[2.0]-CI (abandoned)

dkv.web.vida[2.1]-CI



Azure DevOps dkvnet / CPC_VIDA / Pipelines / Releases / dkv.web.vida[2.1]-CD

All pipelines > dkv.web.vida[2.1]-CD

Pipeline Tasks Variables Retention Options History

Variables

Variable groups

Predefined variables

Filter by keywords

Scope

Name	Value	Scope
API_AGENTE_ACTIVADO	https://apim-dev.dkvseguros.com/cpc/authmediadores/api/v1/Agente	DEV
API_AGENTE_ACTIVADO	https://apim-pre.dkvseguros.com/cpc/authmediadores/api/v1/Agente	UAT
API_AGENTE_ACTIVADO	https://apim-pro.dkvseguros.com/cpc/authmediadores/api/v1/Agente	PRO
API_CONTRATACION	https://apim-dev.dkvseguros.com/cpc/contratacion/generic/v1	DEV
API_CONTRATACION	https://apim-pre.dkvseguros.com/cpc/contratacion/generic/v1	UAT
API_CONTRATACION	https://apim-pro.dkvseguros.com/cpc/contratacion/generic/v1	PRO
API_DATOS_AUX	https://apim-dev.dkvseguros.com/cpc/datosauxiliares/v1	DEV
API_DATOS_AUX	https://apim-pre.dkvseguros.com/cpc/datosauxiliares/v1	UAT
API_DATOS_AUX	https://apim-pro.dkvseguros.com/cpc/datosauxiliares/v1	PRO
API_DOCUMENTACION	https://apim-dev.dkvseguros.com/cpc/documentacion/v3	DEV
API_DOCUMENTACION	https://apim-pre.dkvseguros.com/cpc/documentacion/v3	UAT
API_DOCUMENTACION	https://apim-pro.dkvseguros.com/cpc/documentacion/v3	PRO
API_EMAIL	https://apim-dev.dkvseguros.com/cpc/emailsimulacion/v1	DEV
API_EMAIL	https://apim-pre.dkvseguros.com/cpc/emailsimulacion/v1	UAT
API_EMAIL	https://apim-pro.dkvseguros.com/cpc/emailsimulacion/v1	PRO
API_LEADS	https://apim-dev.dkvseguros.com/cpc/ciclovida/comercial/api/Leads	DEV
API_LEADS	https://apim-pre.dkvseguros.com/cpc/ciclovida/comercial/api/Leads	UAT
API_LEADS	https://apim-pro.dkvseguros.com/cpc/ciclovida/comercial/api/Leads	PRO
API_NORMALIZACION	https://apim-dev.dkvseguros.com/sts/normalizacion/Api/v2	DEV
API_NORMALIZACION	https://apim-pre.dkvseguros.com/sts/normalizacion/Api/v2	UAT
API_NORMALIZACION	https://apim-pro.dkvseguros.com/sts/normalizacion/Api/v2	PRO

Azure DevOps dkvnrt / CPC_VIDA / Pipelines / Releases / dkv.web.vida[2.1]-CD

Search

Save Create release View releases

CPC_VIDA +

Overview Boards Repos Pipelines Pipelines Environments Releases Library Task groups Deployment groups Test Plans Artifacts

All pipelines > dkv.web.vida[2.1]-CD

Pipeline Tasks Variables Retention Options History

Pipeline variables

Variable groups

Predefined variables

Filter by keywords

Release

Name	Value	Scope
containerRegistry	dkvacr01.azurecr.io	Release
imageRepository	dkv.contratacion.web	Release
k8sApplication	dkv-contratacion-v2-web	Release
k8sContainer	\$(containerRegistry)/\$(imageRepository):\$(Build.BuildId).\$(Build.B...	Release
k8sCorsAllowHeaders	Host,Content-Type,Ocp-Apim-Subscription-Key,Ocp-Apim-Trace,cli...	Release
k8sHost	\$(serviceName).\$(k8sNamespace).\$(cluster)	Release
k8sImagePullSecret	docker-dkvreembolsos-cred	Release
releaseNumber	\$(Release.ReleaseName)	Release
serviceName	vida-v2	Release
SUBSCRIPTION_KEY_LEAD	30d78578466041bdaba30863f1c16410	Release

+ Add

Azure DevOps dkvnrt / CPC_VIDA / Pipelines / Releases / dkv.web.vida[2.1]-CD

Search

Save Create release View releases

CPC_VIDA +

Overview Boards Repos Pipelines Pipelines Environments Releases Library Task groups Deployment groups Test Plans Artifacts

All pipelines > dkv.web.vida[2.1]-CD

Pipeline Tasks Variables Retention Options History

Pipeline variables

Variable groups

Predefined variables

Filter by keywords

DEV

Name	Value	Scope
API_AGENTE_ACTIVADO	https://apim-dev.dkvseguros.com/cpc/authmediadores/api/v1/Agente	DEV
API_CONTRATACION	https://apim-dev.dkvseguros.com/cpc/contratacion/generic/v1	DEV
API_DATOS_AUX	https://apim-dev.dkvseguros.com/cpc/datosauxiliares/v1	DEV
API_DOCUMENTACION	https://apim-dev.dkvseguros.com/cpc/documentacion/v3	DEV
API_EMAIL	https://apim-dev.dkvseguros.com/cpc/emailsimulacion/v1	DEV
API_LEADS	https://apim-dev.dkvseguros.com/cpc/ciclovida/comercial/api/Leads	DEV
API_NORMALIZACION	https://apim-dev.dkvseguros.com/sts/normalizacion/api/v2	DEV
API_PROYECTOS	https://apim-dev.dkvseguros.com/cpc/proyectos/v3	DEV
API_SOLICITUD	https://apim-dev.dkvseguros.com/cpm/solicitud/v2/Solicitud/v2	DEV
API_STS	https://apim-dev.dkvseguros.com/sts/validaciones/Validacion/v1	DEV
API_TALENTO	https://dev-ocr.talentomobile.com/id/v5	DEV
API_TARIFICAR	https://apim-dev.dkvseguros.com/cpc/tarificacion/v2/api/v1	DEV
API_TARIFICAR_070	https://apim-dev.dkvseguros.com/cpc/tarificacion/v2/api/v1	DEV
AUTH_TALENTO	e2f1bbbf-a3a0-43e4-b8fd-307e3cbbd017	DEV
authSettings.authority	https://idsrv4.dev.kubedev01.dkvnrt.com	DEV
authSettings.clientId	dkv-contratacion-web.dev	DEV
authSettings.scope	dkv-contexto-api.dev	DEV
CLIENT_ID	CONTRATACION_DEV	DEV
cluster	kubedev01.dkvnrt.com	DEV
HOST	apim-dev.dkvseguros.com	DEV
k8sCorsAllowDomain	https://comercial-v3.dev.kubedev01.dkvnrt.com/	DEV